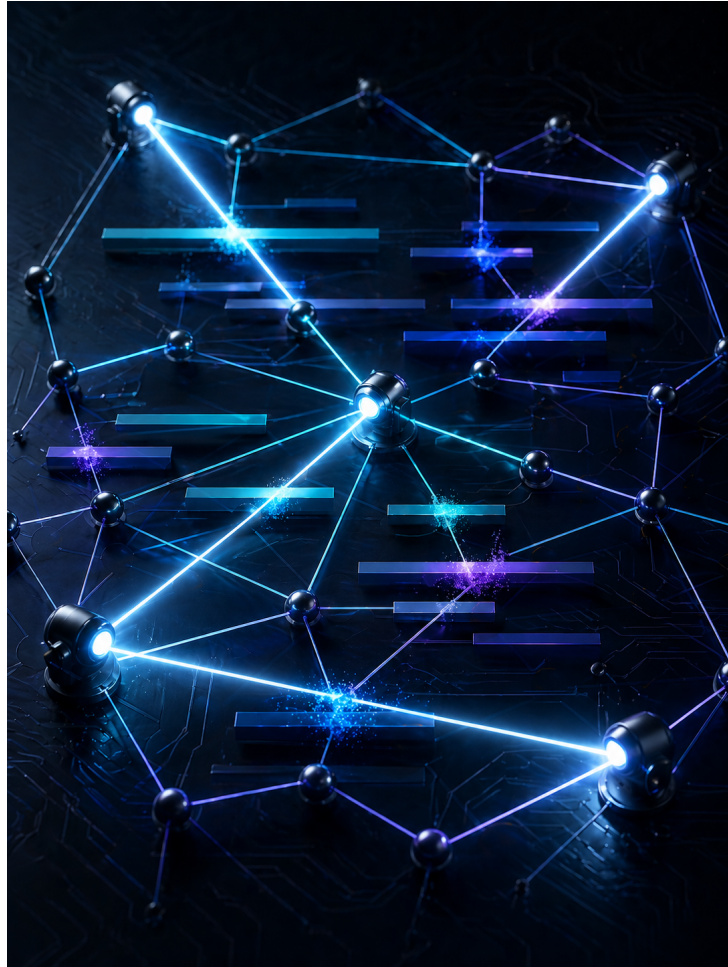




Práctica 3: Algoritmos Voraces

(Greedy)

ALGORÍTMICA | GRUPO C1-3

Problema 1

Omar Segura Arévalo
Marvin Augusto Fonseca Zamora

Problema 2

Jaime Espín Rodríguez
Francisco González Bailón
Alejandro Moreno Millán

Índice

Problema 1: Análisis Químico	1
Enunciado	1
Desarrollo de Algoritmo Óptimo	1
Definición del Flujo de Datos	1
Lógica Principal del Algoritmo	1
Notas Adicionales sobre Decisiones de Diseño	2
Demostración de Validez	2
Desarrollo del Algoritmo Subóptimo	3
Análisis de Eficiencia	3
Algoritmo Voraz - Eficiencia Teórica (Óptimo)	3
Algoritmo Voraz - Eficiencia Teórica (Subóptimo)	4
Análisis Empírico	4
Conclusiones	8
Problema 2: Cámaras de Vigilancia	8
Enunciado y Descripción del Problema	8
Planteamiento y División del Problema	8
Desarrollo del Algoritmo Óptimo (Fuerza Bruta)	8
Lógica Principal del Algoritmo	8
Análisis de Eficiencia Teórica	9
Desarrollo del Algoritmo Subóptimo (Voraz)	9
La Esencia del Algoritmo: Selección por Grado Máximo	9
Análisis de Eficiencia Teórica	9
Análisis Empírico: Voraz vs. Fuerza Bruta (Grafos Generales)	10
Tabulación de Resultados	10
Representación Gráfica (Grafos Generales)	12
Desarrollo de Optimización Específica para Árboles (Voraz)	13
Análisis de Eficiencia Teórica (Voraz en Árboles)	13
Análisis Empírico sobre Árboles	13
Representación Gráfica (Árboles)	15
Conclusiones	16

Problema 1: Análisis Químico

Enunciado

“Un técnico de laboratorio quiere averiguar la composición química de un asteroide. Para ello, dispara láseres a distintas frecuencias al asteroide. Cada sustancia si de una lista de n posibles sustancias constituyentes tiene un rango conocido $[l_i, h_i]$ en el que la sustancia reaccionaría al láser (los rangos pueden estar solapados). El objetivo es elegir el conjunto más pequeño de frecuencias $f_1, \dots, f_k$ para que cada sustancia si reaccione a una de estas, es decir, para cada $i = 1, \dots, n$ hay al menos un j con $l_i \leq f_j \leq h_i$.”

Desarrollo de Algoritmo Óptimo

Según el enunciado del problema, nuestro objetivo es minimizar el número total de frecuencias (disparos láser) necesarias para hacer reaccionar un conjunto de sustancias. Dichas sustancias químicas están representadas por los intervalos de frecuencias $[l_i, h_i]$ de radiación electromagnética a la que reaccionan.

Definición del Flujo de Datos

Para acercarnos lo máximo posible a una situación del mundo real, partimos de la premisa de que los datos de entrada nos llegan desordenados. Por lo que recibiremos por referencia un `vector<pair<double,double>>` que representa la secuencia de intervalos de frecuencia, y un `vector<double>` vacío que guardará nuestra solución de secuencia de frecuencias mínimas. Quedándonos la siguiente cabecera para nuestra función principal:

```
int minFrecuencias(const vector<pair<double,double>> &intervalos, vector<double>
&frecuencias);
```

Lógica Principal del Algoritmo

Ordenamiento de los Datos El primer paso que nos pareció obvio y necesario, sobre todo teniendo en cuenta que nos basamos en la técnica de diseño de algoritmo voraz, para tomar decisiones óptimas locales es ordenar estos datos. Si intentáramos encontrar las intersecciones a fuerza bruta sin un orden previo, la complejidad se dispararía. Con lo cual, delegamos este ordenamiento de la manera que nos pareció más elegante: un contenedor `set` inicializandolo de la siguiente manera:

```
set<pair<double,double>> subconjuntos(intervalos.begin(), intervalos.end());
```

El coste de insertar un elemento en un `set` es de $\log n$, por lo que al tener n datos esta decisión de diseño establece directamente la cota mínima del orden de eficiencia de nuestro algoritmo en $O(n \log n)$. A partir de aquí, el procesamiento de los intervalos no superará esta complejidad.

La Esencia del Algoritmo: Búsqueda Voraz de Intersecciones. Nuestra estrategia principal se basa en que para minimizar los disparos, debemos agrupar la mayor cantidad de sustancias que compartan un rango de frecuencias válido y disparar en el punto límite que las satisfaga a todas.

Paso A: El bucle de Control de minFrecuencias El núcleo del algoritmo principal es un bucle `while` que se ejecutará mientras queden sustancias por procesar en nuestro `set`. En cada iteración, delegamos la tarea pesada a nuestra función auxiliar `encuentraIntersecciones`, pasándole dos cosas por referencia:

1. Todo el conjunto de intervalos restantes: `set<pair<double,double>> &subconjunto`
2. Una `cota_sup` inicial, que siempre será el límite superior del primer intervalo que estamos evaluando en ese momento (`subconjuntos.begin()->second`).

Esta función nos devuelve la frecuencia exacta donde debemos disparar. La guardamos en nuestro vector de resultados `frecuencias` y el bucle repite el proceso con los intervalos que hayan sobrado.

Paso B: Encontrando Todas las Intersecciones de un intervalo encuentraIntersecciones Esta función evalúa qué tantos intervalos consecutivos se solapan y calcula el punto exacto de disparo. Actúa de la siguiente manera:

- **Caso Base:** Si en el conjunto queda menos de 2 elementos, significa que no hay nada más con qué comparar. Limpiamos el conjunto para romper el bucle principal y devolvemos la `cota_sup` actual. Esa es nuestra frecuencia de disparo.
- **Caso Recursivo:** Si hay más elementos, tomamos un iterador al siguiente elemento en el set y comparamos su inicio con nuestra `cota_sup` actual. Aquí se nos abren dos caminos en función de la presencia o ausencia de intersecciones:
 - **En ausencia:** Si nuestra `cota_sup` es estrictamente menor que el inicio del siguiente intervalo (`cota_sup < siguiente->first`), significa que no el intervalo actual no tiene intersecciones con ningún otro intervalo al estar ordenados, por lo que sabemos que hemos encontrado nuestro límite. Borramos el elemento actual del set y devolvemos la `cota_sup` que traíamos.
 - **En presencia:** Si los intervalos sí se solapan, tenemos que asegurarnos de que nuestro disparo alcance a ambos. Para ello, actualizamos nuestra `cota_sup` tomando el mínimo entre esta y el límite superior del nuevo intervalo (`siguiente->second`). Una vez ajustada la nueva cota más estricta, borramos el intervalo actual que acabamos de procesar y hacemos una llamada recursiva, pasando el set (ahora sin el primer elemento) y nuestra nueva `cota_sup` ajustada para que se compare con el siguiente de la lista.

Notas Adicionales sobre Decisiones de Diseño

- **Recursividad vs. Iteración en encuentraIntersecciones:** Decidimos implementar esta función de manera recursiva por mera elegancia resolutive y legibilidad del código, siendo plenamente conscientes de que las pruebas no iban a tratar con conjuntos de datos extremos. Cabe recalcar, sin embargo, que de no haber sido el caso, tendríamos que haber diseñado la función de forma iterativa para evitar un posible Stack Overflow causada por el apilamiento masivo en cada llamada recursiva.
- **La elección de set frente a multiset:** Nos decantamos por inicializar un `set` clásico porque nos conviene eliminar los elementos repetidos desde el principio. En el contexto de nuestro problema, si hay dos o más sustancias con exactamente el mismo intervalo de reacción, basta con evaluar uno de ellos; procesar intervalos idénticos duplicados con un `multiset` no aportaría nada a la lógica de la búsqueda de intersecciones entre intervalos y solo nos haría gastar recursos.

Demostración de Validez

Partimos de un conjunto de n sustancias, cada una con un intervalo de reacción $I_i = [l_i, h_i]$. Queremos encontrar un conjunto mínimo de frecuencias $F = \{f_1, f_2, \dots, f_k\}$ tal que $\forall I_i, \exists$ al menos un $f_j \in I_i$.

Nuestro Algoritmo Voraz Actualmente nuestro algoritmo ordena los intervalos de manera ascendente por su l_i , luego evalúa los intervalos solapados y coloca siempre el láser f_1 (primer disparo) en el valor más a la derecha posible de la intersección actual, siendo entonces $f_1 = \min(h_i)$ de los intervalos que se solapan.

Demostración por reducción al absurdo Sea $I_1 = [l_1, h_1]$ el intervalo más a la izquierda sin cubrir. Toda solución válida deberá disparar en algún $f \in [l_1, h_1]$ para cubrirlo.

- Nuestro algoritmo elige como punto de disparo p^* el valor mínimo de los extremos derechos de los intervalos que se solapan en la iteración actual. Es decir, $p^* = \min\{h_i \mid I_i \in G\}$, siendo G el conjunto de intervalos solapados.

Sea S^* una solución óptima que dispare en algún $p \in [l_1, h_1]$. Si $p \neq p^*$, construimos S' reemplazando p por p^* .

- Como p^* es el mínimo derecho del grupo, $p^* \leq h_1$, así que $p^* \in [l_1, h_1]$
- Entonces todo $I_i = [l_i, h_i]$ cubierto por p con $l_i \leq p \leq h_i$ satisface $l_i \leq p^* \leq h_i$ porque:
 - $l_i \leq p^*$: como el intervalo está en el grupo solapado, $l_i \leq cota_sup$ en algún paso.
 - $p^* \leq h_i$: por definición, p^* es el mínimo de los extremos derechos del grupo.

Por tanto, S' cubre al menos lo mismo que S^* con el mismo número de disparos. $|S'| = |S^*|$, luego nuestro algoritmo greedy no puede ser subóptimo.

Desarrollo del Algoritmo Subóptimo

Tal y como establece el guión de la práctica, una vez implementado el algoritmo principal, debemos proponer y desarrollar una alternativa para poder compararlos.

La primera decisión que debemos tomar es el tipo de enfoque a utilizar. Optamos por mantener la filosofía voraz, pero aplicando un criterio de selección distinto al que ya teníamos desarrollado para observar cómo afecta esta decisión al resultado.

La idea fue simple: cambiar únicamente la estrategia de dónde disparar el láser. Mientras que el algoritmo anterior (el óptimo) evalúa las cotas superiores para disparar lo más a la derecha posible de la intersección, en este nuevo algoritmo (subóptimo) la decisión voraz será disparar exactamente al inicio de cada intervalo no cubierto.

Para ello hemos creado la función `int minFrecuenciasGreedySuboptimo(vector<pair<double, double>>& intervalos, vector<double>& frecuencias)`, la cual recibe la lista de los intervalos de las sustancias y un vector por referencia donde iremos insertando las frecuencias elegidas. Además, devuelve como entero el número total de disparos necesarios.

El primer paso del algoritmo es ordenar los intervalos por su inicio. Para conseguirlo, utilizamos la función `std::sort` definida en la librería `<algorithm>`. Esta función requiere que le indiquemos cómo debe ordenar los datos, por lo que hemos implementado la siguiente función para que compare:

```
bool ordenarInicio(const pair<double, double>& a, const pair<double, double>& b){
    if(a.first == b.first){ // Si el primer componente es igual, miramos el segundo
        return a.second < b.second;
    }

    return a.first < b.first;
}
```

Esta función auxiliar simplemente compara el primer elemento (el inicio del intervalo) de dos pares para devolver el menor. En caso de que ambos intervalos comiencen exactamente en el mismo punto, desempata evaluando el segundo elemento (el final del intervalo).

Una vez tenemos los datos ordenados, iteramos sobre cada uno de los intervalos. Para cada sustancia, debemos comprobar si alguna de las frecuencias que ya tenemos guardadas sirve para hacerla reaccionar. Si está cubierta, no hacemos nada; si, por el contrario, no existe ninguna frecuencia guardada que sirva para hacerla reaccionar, tomamos la decisión voraz y disparamos al principio del intervalo.

Toda esta lógica queda reflejada en el siguiente fragmento:

```
for(const auto& intervalo : intervalos){
    bool alcanzado = false;

    // Vemos si tenemos una frecuencia que sirva
    for(int i = 0; i < frecuencias.size() && !alcanzado; i++){
        const auto& f = frecuencias[i];
        if(f >= intervalo.first && f <= intervalo.second) alcanzado = true;
    }

    // Si no lo hemos alcanzado con ninguna frecuencia, disparamos al inicio
    if(!alcanzado){
        frecuencias.push_back(intervalo.first);
    }
}
```

Ánàlisis de Eficiencia

Algoritmo Voraz - Eficiencia Teórica (Óptimo)

Nuestro algoritmo comienza en la función `minFrecuencias` donde introducimos `n` intervalos en un `set<pair<double,double>>`. Internamente, C++ gestiona el `set` como un árbol binario de búsqueda balanceado, por lo que la inserción de los `n` elementos tiene una eficiencia $O(n \log n)$.

Posteriormente, entramos en el núcleo del algoritmo al realizar el bucle `while`, para analizarlo tendremos que ver qué hacemos dentro de la función `encuentraIntersecciones`. Dentro de dicha función, solo hacemos un `subconjunto.erase(subconjunto.begin())`, es decir, solamente usa un intervalo y lo elimina; eliminar el primer elemento de un `set` tiene una eficiencia $O(\log n)$, por lo que si en el bucle `while` llamamos recursivamente para los n intervalos, el coste de esta fase es $O(n) \cdot O(\log n) = O(n \log n)$. Si sumamos el coste de la función `encuentraIntersecciones` con el coste de la inserción en el `set` nos queda que la eficiencia de nuestro algoritmo voraz es $O(n \log n) + O(n \log n) = \mathbf{O(n \log n)}$.

Algoritmo Voraz - Eficiencia Teórica (Subóptimo)

Empezamos el análisis en la ordenación. En este caso hemos preferido usar `std::sort` para ordenar por el inicio del intervalo. El coste estándar de hacer esta operación es $O(n \log n)$.

Posteriormente tenemos un doble bucle:

```
for(const auto& intervalo : intervalos){
    bool alcanzado = false;
    for(int i = 0; i < frecuencias.size() && !alcanzado; i++){
        const auto& f = frecuencias[i];
        if(f >= intervalo.first && f <= intervalo.second) alcanzado = true;
    }

    //push_back...
}
```

En el mejor de los casos, es decir, que los intervalos se solapen tanto que el vector de frecuencias se mantenga en tamaño 1, el bucle interno tendrá un coste constante $O(1)$, dejando el coste de recorrer todos los intervalos en $O(n)$.

Sin embargo, en el peor de los casos (donde ningún intervalo se solapa y el vector de frecuencias crece hasta tamaño n), el bucle interno iterará hasta el final en cada paso. Esto genera una suma aritmética que nos deja una complejidad cuadrática de $O(n^2)$.

Por lo tanto, si al coste de la ordenación le sumamos el coste del bucle anidado, nos queda que la eficiencia asintótica de este algoritmo es $O(n \log n) + O(n^2) = \mathbf{O(n^2)}$

Análisis Empírico

Para el análisis empírico hemos decidido realizar una función `realizarEstudioEmpirico` que hará uso de las funciones descritas a lo largo del desarrollo del problema. Para realizar las mediciones, hemos utilizado una clase auxiliar (que ya hemos utilizado en prácticas anteriores) para tener acceso al reloj preciso del ordenador y así poder realizar las tomas de tiempo.

Como tenemos que analizar dos algoritmos de diferente eficiencia, hemos prefijado tamaños que se puedan adecuar a ambos, definiendo un vector de la siguiente forma: `vector<int> tams = {1000, 2000, 4000, 8000, 16000, 32000, 64000, 128000};`. Además, para evitar ruidos del sistema, hemos preferido hacer k iteraciones para cada tamaño, para posteriormente realizar la media.

Una vez definida la función, los tamaños y el valor de k , hemos compilado y tomado los tiempos.

Análisis Empírico (óptimo) Para el algoritmo óptimo hemos obtenido los siguientes tiempos:

Nota: Véase el archivo `datosMemoria/algoritmoOptimo.dat` donde están todos los datos en bruto generados por el algoritmo.

n	Tiempo
1000	0.00039126
2000	0.000798587
4000	0.00193449
8000	0.0032115
16000	0.00754194
32000	0.0174643

n	Tiempo
64000	0.0356389
128000	0.0801229

A partir de los datos obtenidos, y utilizando la herramienta Gnuplot, hemos podido realizar la gráfica correspondiente:

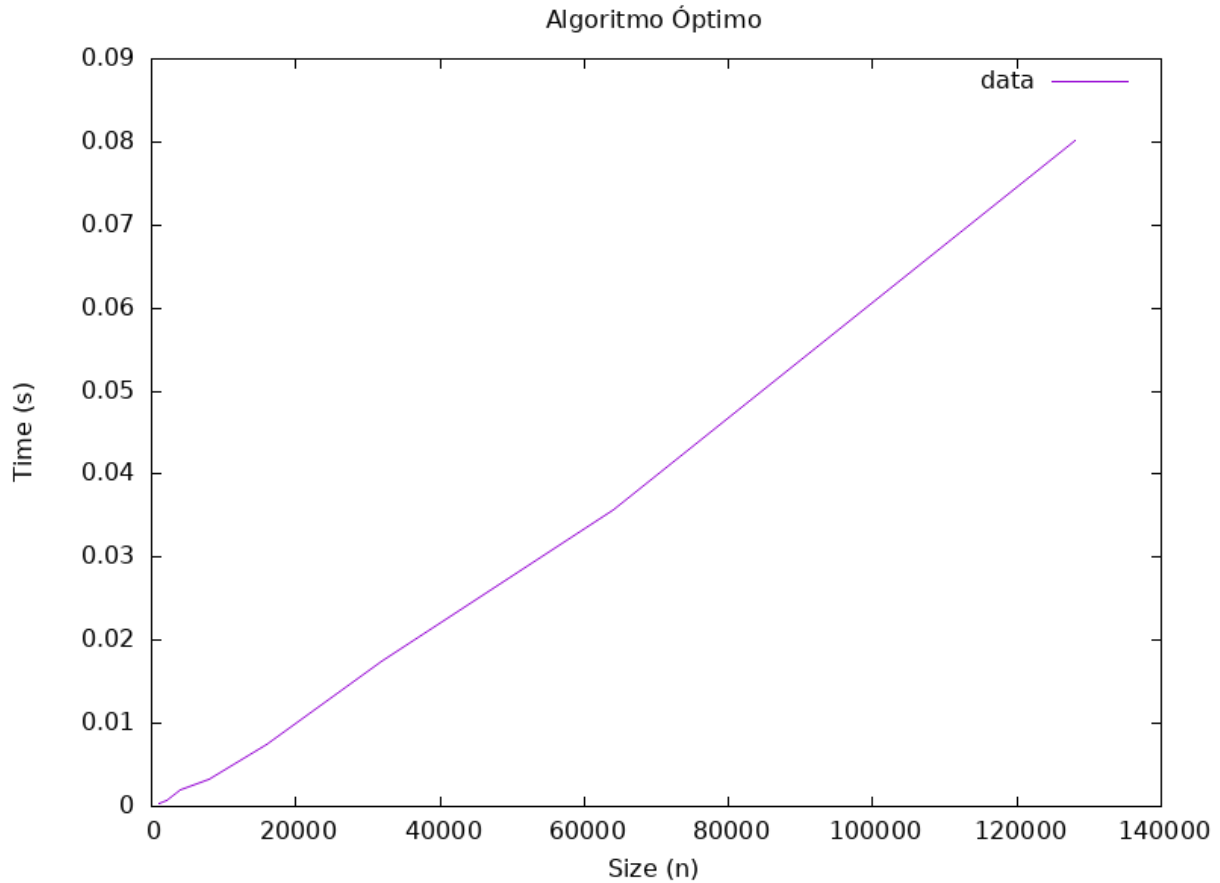


Figura 1: Gráfica del algoritmo Voraz Óptimo

En esta gráfica se representa el crecimiento del tiempo de ejecución del algoritmo óptimo (en escala lineal). Como se puede apreciar, el comportamiento del tiempo es extremadamente contenido; incluso al someter al algoritmo a una carga de 128.000 intervalos, el tiempo de procesamiento se mantiene por debajo de las 0.08 décimas de segundo, demostrando una altísima eficiencia en la práctica.

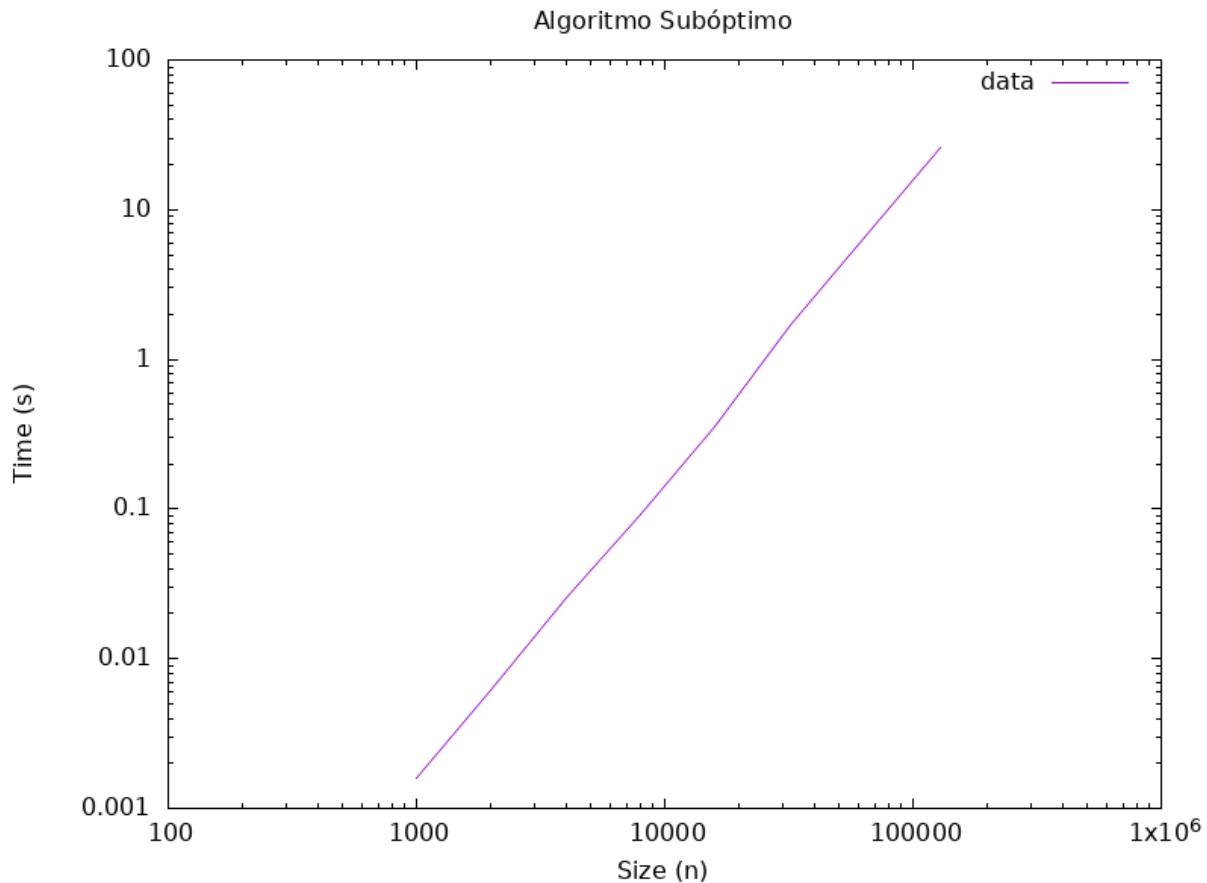
Análisis Empírico (Subóptimo) Para el algoritmo subóptimo hemos obtenido los siguientes tiempos:

NOTA: Véase el archivo `datosMemoria/algoritmoSuboptimo.dat` donde están todos los datos en bruto generados por el algoritmo.

n	Tiempo
1000	0.0015957
2000	0.00608971
4000	0.0257527
8000	0.0925139
16000	0.352638
32000	1.69736
64000	6.50092

n	Tiempo
128000	25.9147

A partir de los datos obtenidos, y utilizando la herramienta Gnuplot, hemos podido realizar la gráfica correspondiente:



NOTA: En esta gráfica se ha empleado escala logarítmica.

A diferencia del algoritmo óptimo, esta gráfica refleja un crecimiento mucho más agresivo del tiempo de ejecución frente al aumento del tamaño de la entrada. Para la misma carga máxima de 128.000 intervalos, el algoritmo requiere casi 26 segundos en finalizar.

Comparación Empírica En la gráfica siguiente se presenta la comparativa empírica del tiempo de ejecución (en segundos) de ambos algoritmos en función del número de intervalos (n). Para facilitar la visualización de ambas curvas y apreciar sus órdenes de magnitud, se ha empleado una escala logarítmica en ambos ejes. A partir de esta representación gráfica, podemos extraer conclusiones que respaldan directamente nuestro análisis teórico previo:

- Algoritmo Voraz Subóptimo (Línea verde): Muestra una pendiente pronunciada y constante. Tiene una mayor inclinación, lo que es el rasgo característico de una función polinómica de mayor grado, lo que corrobora visualmente nuestra cota teórica cuadrática de $O(n^2)$. El efecto es evidente: a medida que el tamaño de la entrada crece, el coste de cómputo se dispara debido a los bucles anidados.
- Algoritmo Voraz Óptimo (Línea morada): Presenta una pendiente mucho más suave. En la escala logarítmica, una complejidad de $O(n \log n)$ se asemeja a una recta con una pendiente cercana a 1. Como se observa en los datos empíricos, el algoritmo escala de manera excelente, resolviendo problemas de hasta 128.000 intervalos en apenas milésimas de segundo gracias a la eficiencia del árbol binario balanceado (`std::set`).

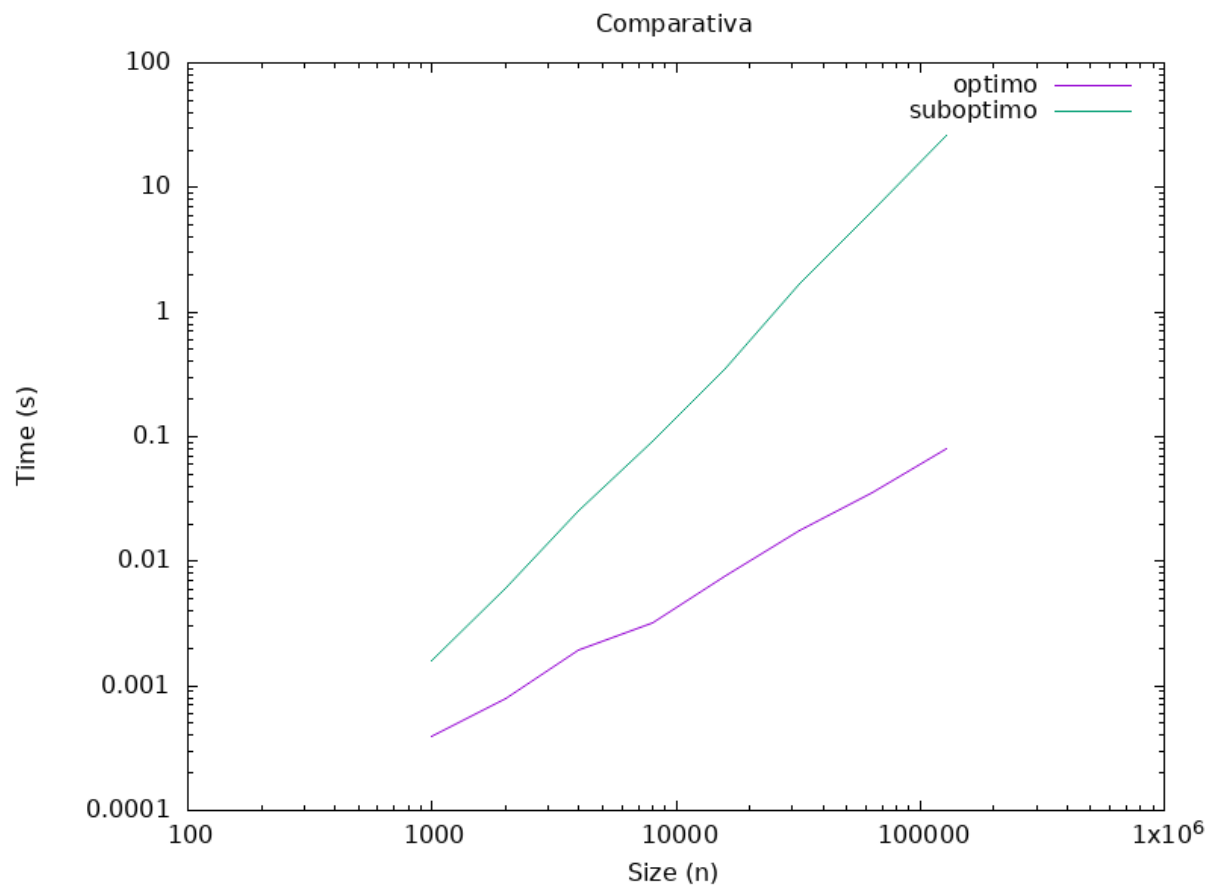


Figura 2: Gráfica comparativa de los tiempos de ejecución del algoritmo Voraz Óptimo frente al Subóptimo

Conclusiones

Tras el análisis realizado sobre el problema del alineamiento de frecuencias láser, podemos extraer las siguientes conclusiones fundamentales:

- Eficacia de la Estrategia Voraz: Se ha demostrado, tanto de forma teórica como práctica, que el enfoque greedy es idóneo para este problema. Sin embargo, la clave del éxito reside en la elección del criterio de selección. Mientras que disparar al inicio de cada intervalo (algoritmo subóptimo) cumple con el objetivo de cubrir todas las sustancias, solo la búsqueda de la intersección máxima hacia el extremo derecho (algoritmo óptimo) garantiza el uso del número mínimo absoluto de disparos.
- Impacto de las Estructuras de Datos: La implementación del algoritmo óptimo mediante el uso de `std::set` (árboles binarios balanceados) ha resultado ser determinante. Esta decisión de diseño ha permitido mantener una complejidad de $O(n \log n)$, marcando una diferencia abismal frente al comportamiento cuadrático $O(n^2)$ del algoritmo subóptimo basado en vectores y búsquedas lineales.

Problema 2: Cámaras de Vigilancia

Enunciado y Descripción del Problema

El objetivo fundamental de este problema es garantizar la seguridad de un complejo arquitectónico formado por una serie de intersecciones y pasillos. Para ello, partimos de la premisa de que una cámara de vigilancia colocada estratégicamente en una intersección es capaz de monitorizar todos los pasillos que convergen en ella.

Nuestra meta es encontrar la solución óptima absoluta. Cabe aclarar que, en el contexto de este problema, el término “óptimo” trasciende la mera eficiencia temporal del algoritmo (tiempo de ejecución); se refiere de manera prioritaria a la consecución de una disposición espacial que requiera instalar el número estrictamente mínimo de cámaras posibles para cubrir la totalidad del recinto.

Planteamiento y División del Problema

Para poder abordar y analizar de forma exhaustiva la naturaleza del problema propuesto, hemos optado por bifurcar nuestro enfoque desarrollando dos soluciones diametralmente opuestas en su concepción:

1. Algoritmo Óptimo (Fuerza Bruta)

- Garantiza la obtención de la solución óptima absoluta mediante la evaluación sistemática de todo el espacio de soluciones posibles.
- Presenta la limitación inherente de un coste computacional extremadamente elevado, volviéndose inmanejable conforme la dimensión del problema escala.

2. Algoritmo Subóptimo (Voraz / Greedy)

- Renuncia a la garantía matemática de hallar la solución óptima absoluta.
- A cambio, ofrece tiempos de ejecución drásticamente inferiores frente a la fuerza bruta.
- Garantiza alcanzar una solución “suficientemente buena” (una aproximación de alta calidad, muy cercana al óptimo real).

Desarrollo del Algoritmo Óptimo (Fuerza Bruta)

Lógica Principal del Algoritmo

El algoritmo de fuerza bruta se cimenta sobre la idea de explorar recursivamente todas las combinaciones posibles referentes a la colocación o no colocación de cámaras en cada una de las intersecciones disponibles.

El esquema de funcionamiento es el siguiente: 1. En la intersección actual bajo evaluación, tomamos una decisión binaria: colocar una cámara (1) o prescindir de ella (0). 2. Efectuamos una llamada recursiva hacia la siguiente intersección, ramificando el proceso para ambas decisiones. 3. Al alcanzar el caso base de la recursividad (cuando se han tomado decisiones para el conjunto total de intersecciones), procedemos a verificar mediante nuestra matriz de adyacencia si la combinación de cámaras propuesta es factible (es decir, si no deja ningún pasillo sin vigilar). 4. Si la combinación resulta ser válida y, además, mejora la cantidad de cámaras de la mejor solución registrada hasta el momento, procedemos a almacenarla como la nueva solución óptima provisional.

Análisis de Eficiencia Teórica

El diseño de este algoritmo por fuerza bruta fracciona el problema general de tomar n decisiones en 2 subproblemas de tamaño $n - 1$, añadiendo un esfuerzo computacional constante en cada nodo del árbol recursivo. Definiendo el tamaño decreciente de la entrada como m , y estableciendo el inicio de la recursión en $m = n$ intersecciones restantes por evaluar, obtenemos la siguiente ecuación en recurrencia para el número de nodos:

$$T(m) = 2T(m - 1) + c$$

Para resolver dicha ecuación, recurrimos a la **ecuación característica**: 1. Planteamos la recurrencia en su forma canónica: $T(m) - 2T(m - 1) = c$ 2. Buscamos la solución a la ecuación homogénea asociada ($T^{(h)}(m) - 2T^{(h)}(m - 1) = 0$). Su polinomio característico es $r - 2 = 0$, que arroja una única raíz $r = 2$. Por consiguiente, deducimos que $T^{(h)}(m) = c_1 \cdot 2^m$. 3. A continuación, buscamos una solución particular para la constante c . Dado que 1 no constituye una raíz de la ecuación homogénea, proponemos $T^{(p)}(m) = c_2$. Sustituyendo en la original: $c_2 - 2c_2 = c \implies c_2 = -c$. 4. Ensamblando ambas partes, la solución general de nuestra recurrencia queda definida como:

$$T(m) = c_1 \cdot 2^m - c$$

A la luz de estos resultados, comprobamos que el árbol de llamadas recursivas exhibe un crecimiento de orden $\mathcal{O}(2^n)$. No obstante, al alcanzar las 2^n hojas del árbol (el escenario donde se han consumado las decisiones para los n nodos, resultando en $m = 0$), el algoritmo invoca la función de validación `Factiblefb()`. Esta rutina escudriña la mitad superior de la matriz de adyacencia en busca de aristas para verificar su cobertura, operación que conlleva un coste inherente de $\mathcal{O}(n^2)$.

Dado que este coste terminal se incurre 2^n veces, concluimos que la **complejidad temporal asintótica del algoritmo** de 2^n).

Desarrollo del Algoritmo Subóptimo (Voraz)

La Esencia del Algoritmo: Selección por Grado Máximo

Nuestra estrategia voraz abraza un enfoque heurístico: en cada iteración del proceso, la decisión óptima local consistirá en seleccionar siempre aquella intersección que sea capaz de abarcar la mayor cantidad de pasillos ciegos. En términos de grafos, esto se traduce en elegir invariablemente el nodo que ostente el mayor grado dentro de la submatriz restante.

El flujo de ejecución de la función principal `greedy()` es el siguiente: 1. Instanciamos una matriz auxiliar `aux` a la cual copiamos los datos originales, preservando así la integridad de la información inicial. 2. Iniciamos un bucle de control que persistirá en su ejecución mientras el sistema detecte pasillos desprovistos de vigilancia. 3. Se realiza un escaneo completo de la matriz para aislar la intersección con el mayor número de conexiones activas (el nodo de mayor grado). 4. Registramos dicha intersección como elemento constituyente de nuestra solución. 5. Procedemos a marcar como “vigilados” todos los pasillos incidentes a la intersección seleccionada (efectuando un borrado lógico de su correspondiente fila y columna en la matriz `aux`).

Análisis de Eficiencia Teórica

Desglosando internamente el núcleo de la función `greedy()`, identificamos un bucle `while` que alberga tres operaciones fundamentales:

1. **El bucle de control while**: En el escenario más desfavorable, nos veríamos obligados a instalar una cámara por cada intersección existente, limitando el número máximo de iteraciones a n .
2. **Búsqueda de pasillos** `quedanPasillosSinVigilar(aux)`: Efectúa un barrido exhaustivo sobre la totalidad de la matriz $n \times n$ en busca de aristas remanentes, lo cual impone un coste de $\mathcal{O}(n^2)$.
3. **Cálculo del máximo grado** `verticeMayorGrado(aux)`: Recorre íntegramente la matriz $n \times n$ para cuantificar las incidencias de cada vértice. Este proceso conlleva un coste de $\mathcal{O}(n^2)$.
4. **Actualización de estado** `marcarPasillosVigilados(aux, indice)`: Invalida (asigna a 0) todos los elementos pertenecientes a la fila y columna del vértice elegido, resultando en un coste lineal de $\mathcal{O}(n)$.

Agrupando los componentes iterativos, el coste computacional por cada ciclo asciende a $\mathcal{O}(n^2) + \mathcal{O}(n^2) + \mathcal{O}(n) = \mathcal{O}(n^2)$. Al multiplicar este esfuerzo por la cota superior de iteraciones (n), determinamos que la **complejidad temporal del algoritmo voraz se asienta en $\mathcal{O}(n^3)$** .

Análisis Empírico: Voraz vs. Fuerza Bruta (Grafos Generales)

Con el propósito de validar la viabilidad real de nuestro algoritmo Voraz y cuantificar su divergencia respecto a la solución Óptima absoluta, resulta imperativo someterlos a una confrontación empírica. Nuestro análisis teórico nos advierte que la Fuerza Bruta despliega un crecimiento exponencial $\mathcal{O}(n^2 \cdot 2^n)$, lo cual predice un punto crítico (alrededor de $n \approx 30$) donde la capacidad de cómputo colapsará impidiendo obtener una respuesta en un lapso razonable. En contraste, el comportamiento polinómico $\mathcal{O}(n^3)$ del Voraz le otorga la facultad de resolver configuraciones de miles de pasillos en ínfimas fracciones de segundo.

Para materializar esta comparativa y vislumbrar tanto el abismo temporal como el margen de suboptimalidad de la heurística Greedy, hemos recopilado un conjunto de mediciones de campo que desglosamos a continuación.

Tabulación de Resultados

Para la recolección de los datos empíricos, procedimos a compilar nuestro código fuente en C++ (`camaras-fb.cpp`), prestando especial atención a la habilitación de los correspondientes flags de optimización del compilador. El banco de pruebas se ha alimentado empleando el generador automático de grafos integrado en el propio código, barriendo el espectro de tamaños desde $n = 20$ hasta $n = 28$ intersecciones.

El protocolo para cada instancia evaluada fue el siguiente: - Se sintetizó una matriz de adyacencia aleatoria, manteniendo constante la probabilidad de inserción de aristas. - Se cronometró y registró el desempeño de la función `greedy()`, anotando tanto su tiempo de ejecución como la cantidad de cámaras dictaminadas. - Acto seguido, se replicó el mismo proceso de medición para la llamada a la función óptima `fb_recursivo(0)`.

Intersecciones (n)	Cámaras (Voraz)	Cámaras (Fuerza Bruta)	Tiempo Voraz (s)	Tiempo Fuerza Bruta (s)
20	12	11	0.000089	0.1118
22	14	14	0.000134	0.4398
24	17	16	0.000228	1.8788
25	18	18	0.000183	3.4564
26	18	17	0.000189	7.1207
27	18	18	0.000236	15.4157
28	21	20	0.000511	28.7832

De la atenta lectura de esta tabla, podemos destilar conclusiones reveladoras: 1. La estrategia Voraz es capaz de impactar con éxito en la solución óptima en repetidas ocasiones; sin embargo, adolece de una tendencia a sobredimensionar la infraestructura instalando entre 1 y 2 cámaras superfluas al enfrentarse a grafos de notable complejidad. 2. Resulta demoledor observar cómo, para un modesto escenario de $n = 28$, la Fuerza Bruta consume la friolera de casi 29 segundos de procesamiento. Simultáneamente, el algoritmo Voraz resuelve el mismo dilema en unos irrisorios 0.000511 segundos. Constituye un argumento empírico irrefutable a favor del empleo de heurísticas voraces al lidiar con la intratabilidad de los problemas NP-Complejos en entornos reales.

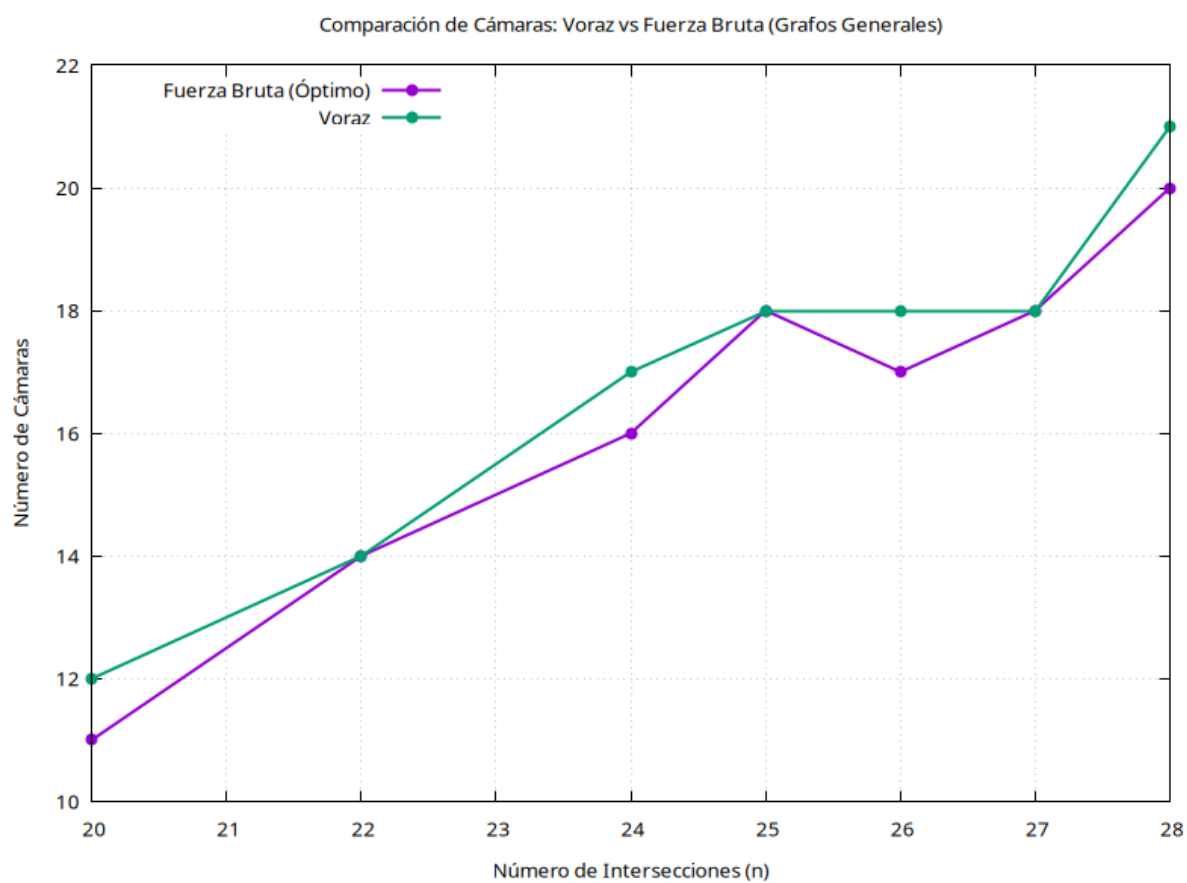
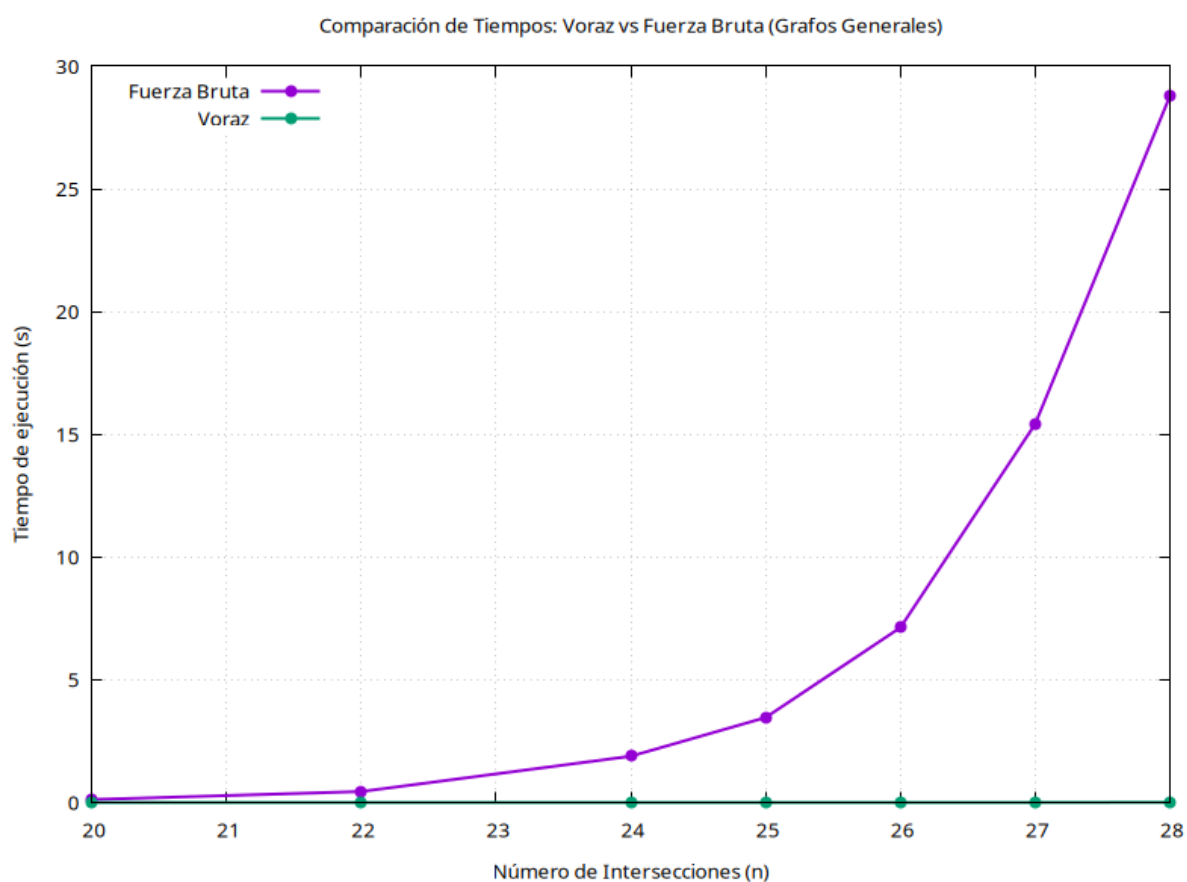
A modo de refuerzo experimental y para ampliar nuestro espacio muestral, adjuntamos a continuación una segunda batería de pruebas ejecutada en el equipo de nuestro compañero Fran:

Intersecciones (n)	Cámaras (Voraz)	Cámaras (Fuerza Bruta)	Tiempo Voraz (s)	Tiempo Fuerza Bruta (s)
20	11	11	0.000221	0.2004
22	15	15	0.000371	0.5047
24	16	16	0.000420	1.9274

Intersecciones (n)	Cámaras (Voraz)	Cámaras (Fuerza Bruta)	Tiempo Voraz (s)	Tiempo Fuerza Bruta (s)
25	16	16	0.000376	3.8893
26	18	18	0.000515	7.5035
27	19	18	0.000563	16.850
28	19	18	0.000552	32.495

La corroboración es evidente: en los umbrales de $n = 27$ y $n = 28$, la heurística voraz prescribe 19 cámaras, rebasando el óptimo absoluto de 18. Queda atestiguado, por tanto, que la heurística no detenta la infalibilidad absoluta. Paralelamente, presenciamos la materialización de la explosión combinatoria, observando un tiempo de ejecución que se amolda a nuestras proyecciones previas (superando la barrera de los 30 segundos para $n = 28$). Por su parte, la latencia del algoritmo Voraz se mantiene imperturbable en el orden de los microsegundos, haciendo gala de una eficiencia técnica excepcional. Podemos dictaminar firmemente que nuestro algoritmo sacrifica una fracción verdaderamente nimia de precisión resolutive como tributo para alcanzar una contracción asombrosa en los tiempos de cómputo.

Representación Gráfica (Grafos Generales)



Desarrollo de Optimización Específica para Árboles (Voraz)

Es de vital importancia matizar que la variante algorítmica que se expone a continuación ha sido concebida de manera exclusiva para operar sobre grafos acíclicos conexos (árboles). Su aplicación sobre grafos de carácter general, susceptibles de contener ciclos en su topología, derivará en comportamientos anómalos y resoluciones incorrectas. Por esta razón de peso, las representaciones gráficas del epígrafe anterior circunscriben su comparativa al Algoritmo Voraz Genérico frente a la Fuerza Bruta. En la presente sección, someteremos a escrutinio a la tríada de algoritmos de manera estricta bajo el dominio de los árboles.

Nuestra aproximación voraz refinada para árboles basa su *modus operandi* en el rastreo de nodos hoja (vértices de grado uno). Una vez localizada una hoja, la heurística toma la decisión irrevocable de emplazar la cámara en el vértice progenitor (el nodo padre). Esta maniobra táctica asegura irremediablemente la cobertura del nodo hoja, de su padre, así como de cualesquiera otros nodos hermanos o ascendientes directos del padre, maximizando el área de influencia de cada dispositivo.

El diagrama de flujo de este mecanismo se detalla a continuación: 1. Generamos una matriz auxiliar **aux** clonando la matriz origen, protegiendo así nuestro modelo de datos. 2. Instanciamos un bucle iterativo que mantendrá la ejecución activa en tanto pervivan pasillos ciegos no vigilados. 3. Penetramos la matriz mediante una exploración focalizada en identificar una hoja, de la cual, acto seguido, derivamos su nodo ascendiente (el padre). 4. Anexionamos dicho nodo padre a nuestro sumatorio de la solución final. 5. Sancionamos como “vigilados” todos los tramos adyacentes a esta nueva instalación, materializando este paso mediante la purga sistemática de la fila y columna pertinentes en la matriz **aux**.

Análisis de Eficiencia Teórica (Voraz en Árboles)

Incursionando en las entrañas de la función `greedyArbol()`, discriminamos el siguiente desglose de operaciones: 1. **Bucle de ejecución while**: Conservando el patrón del voraz genérico, se restringe a un máximo de n iteraciones. 2. **Auditoría de pasillos** `quedanPasillosSinVigilar(aux)`: Ejecuta una lectura integral de la matriz, reportando un coste de $\mathcal{O}(n^2)$. 3. **Localización de hojas** `BuscamosHoja(aux)`: Explora el espacio matricial tabulando el grado de conexión de cada intersección. Su ejecución se interrumpe prematuramente devolviendo el nodo padre en el instante en que halla una incidencia de grado uno. En el umbral del peor de los casos, obligaría a procesar la matriz completa, implicando una penalización de $\mathcal{O}(n^2)$. 4. **Consolidación** `marcarPasillosVigilados(aux, padre_hoja)`: Anula, mediante reasignación a 0, los componentes ortogonales del vértice afectado. Coste: $\mathcal{O}(n)$.

Al cristalizar el coste iterativo, nos topamos con la expresión $\mathcal{O}(n^2) + \mathcal{O}(n^2) + \mathcal{O}(n) = \mathcal{O}(n^2)$. Considerando la frontera superior de n iteraciones, inferimos que la **complejidad temporal asintótica del algoritmo se preserva en** manteniendo la paridad teórica con su homólogo genérico. Pese a ello, como comprobaremos a continuación, la empiria demuestra que esta variante goza de una velocidad de ejecución ligeramente superior; fenómeno atribuible a la parada temprana de la función `BuscamosHoja`, la cual elude la onerosa necesidad de escrutar el grafo en su totalidad, exigencia insoslayable para la función `verticeMayorGrado`.

Análisis Empírico sobre Árboles

Para llevar a cabo una disección comparativa del desempeño de nuestra tríada algorítmica (Fuerza Bruta, Voraz Genérico y Voraz Específico para Árboles), hemos recurrido al módulo generador de grafos acíclicos integrado en nuestro software.

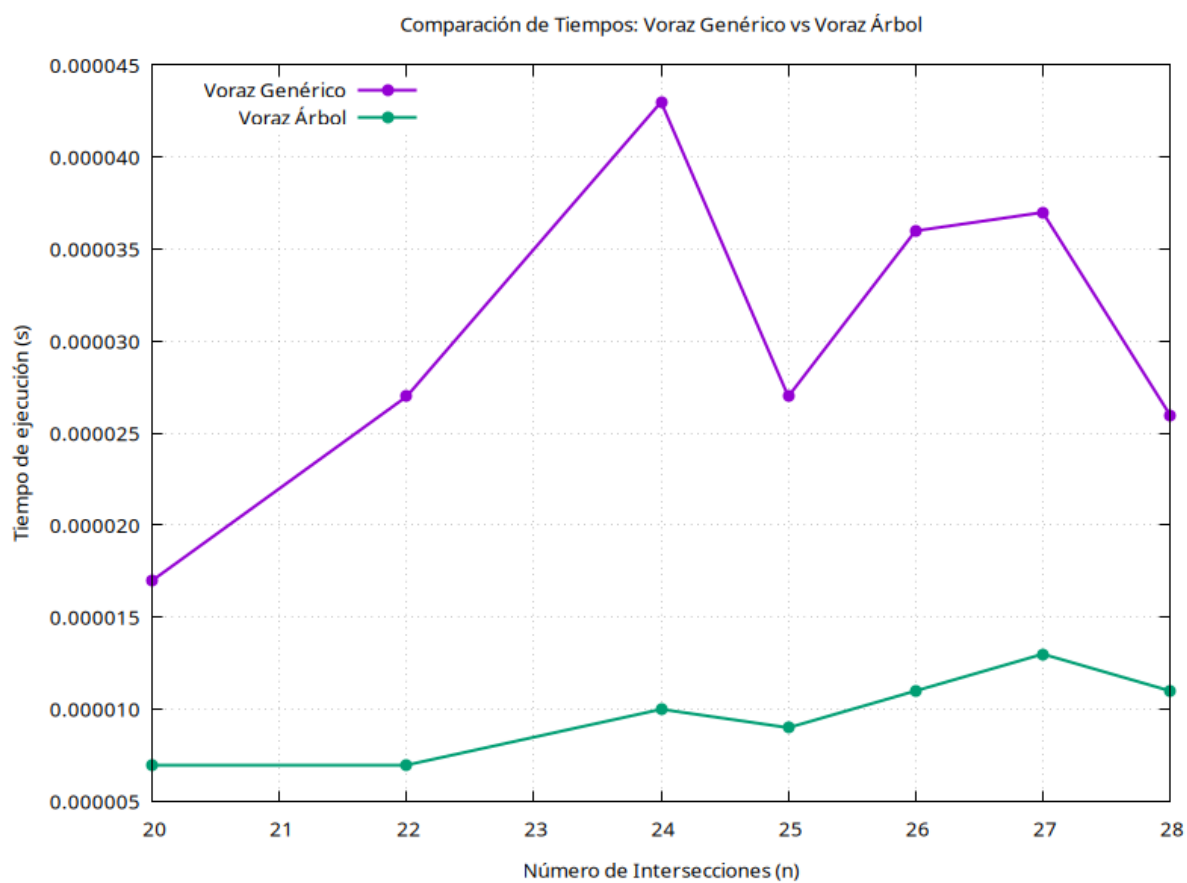
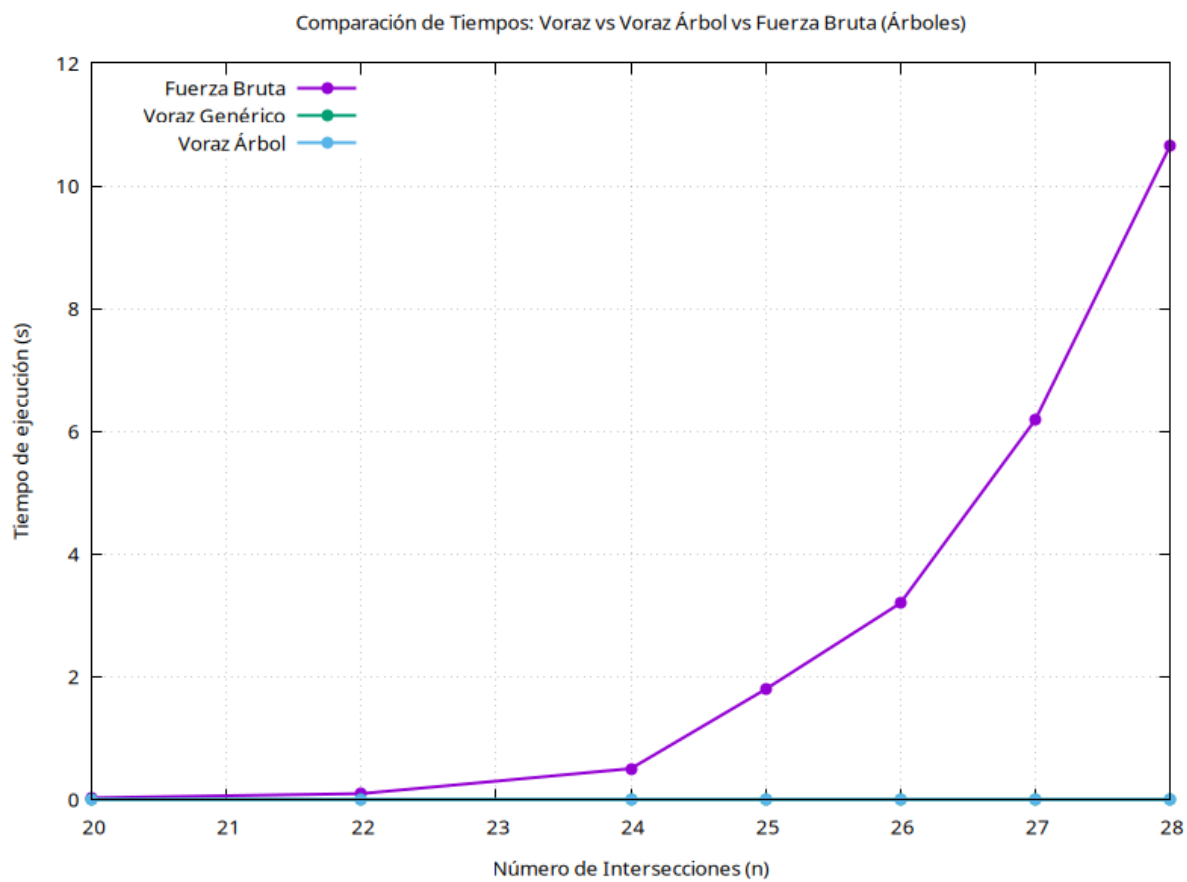
Intersecciones (n)	Cámaras (Voraz Genérico)	Cámaras (Voraz Árbol)	Cámaras (Fuerza Bruta)	Tiempo Voraz Genérico (s)	Tiempo Voraz Árbol (s)	Tiempo FB (s)
20	8	8	8	0.000017	0.000007	0.023677
22	10	9	9	0.000027	0.000007	0.092646
24	11	10	10	0.000043	0.000010	0.500536
25	10	10	10	0.000027	0.000009	1.795116
26	11	11	11	0.000036	0.000011	3.198420
27	12	12	12	0.000037	0.000013	6.182430
28	10	10	10	0.000026	0.000011	10.649700

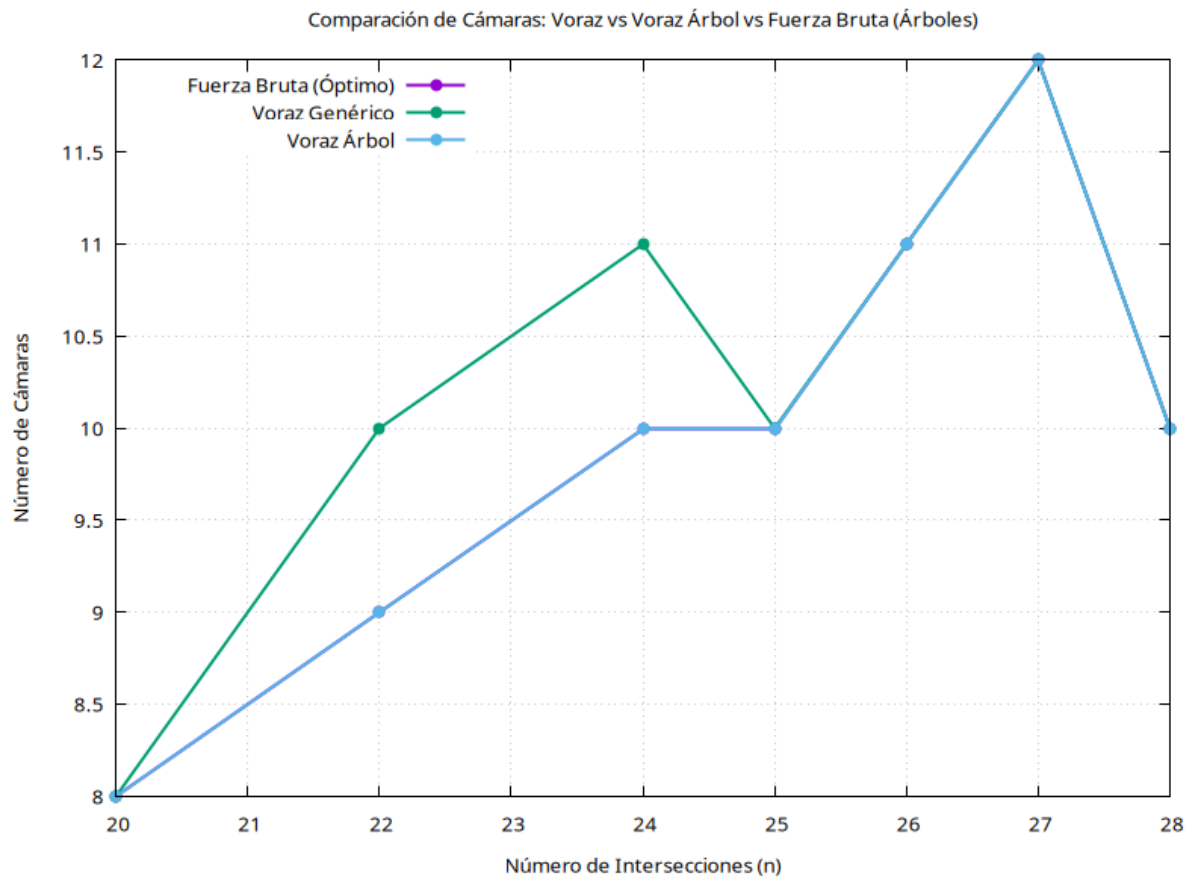
Del minucioso escrutinio de la tabla de recolección de datos, emergen certidumbres irrefutables: 1. El

Algoritmo Voraz Específico para Árboles converge invariablemente hacia la solución óptima absoluta.

Sus dictámenes en cuanto al número de cámaras se alinean con precisión milimétrica frente a los veredictos de la Fuerza Bruta para todas las instancias evaluadas. Este hito es especialmente destacable en aquellos escenarios (como $n = 22$ y $n = 24$) donde la heurística del Voraz genérico flaqueó, prescribiendo infraestructura excedente. 2. En lo concerniente a los tiempos de cómputo, se constata que la **variante voraz especializada en árboles resulta consistentemente más veloz** que la implementación voraz genérica (oscilando entre márgenes del doble e incluso el triple de velocidad procesal). Esta aceleración es fruto directo de la optimización táctica mencionada con anterioridad, que permite la ruptura anticipada del bucle de búsqueda tras el hallazgo de la primera hoja disponible.

Representación Gráfica (Árboles)





Conclusiones

Tras el profundo análisis ejecutado sobre el problema de la distribución de cámaras de vigilancia, nos encontramos en disposición de enunciar las siguientes conclusiones fundamentales:

- **Eficacia y Necesidad de la Estrategia Voraz:** Ha quedado demostrado empírica y teóricamente el colapso inminente de los algoritmos de búsqueda exhaustiva ante problemas de esta naturaleza. La aproximación heurística (Greedy) se erige no como una mera alternativa, sino como la única vía pragmática para abordar el problema en escalas de dimensión real, permutando una fracción asumible de exactitud por una escalabilidad computacional formidable.
- **El Valor de la Especialización Topológica:** La comparativa final nos brinda una lección de incalculable valor sobre la importancia del diseño algorítmico contextualizado. Al explotar una propiedad geométrica intrínseca de los grafos de entrada (la aciclicidad de los árboles), hemos logrado forjar una variante voraz capaz de proporcionar soluciones matemática y estrictamente óptimas, batiendo además en velocidad a su predecesor genérico. Esto nos subraya que una heurística cuidadosamente entonada a las propiedades de los datos es capaz de rivalizar con la perfección teórica de la fuerza bruta sin heredar sus limitaciones.